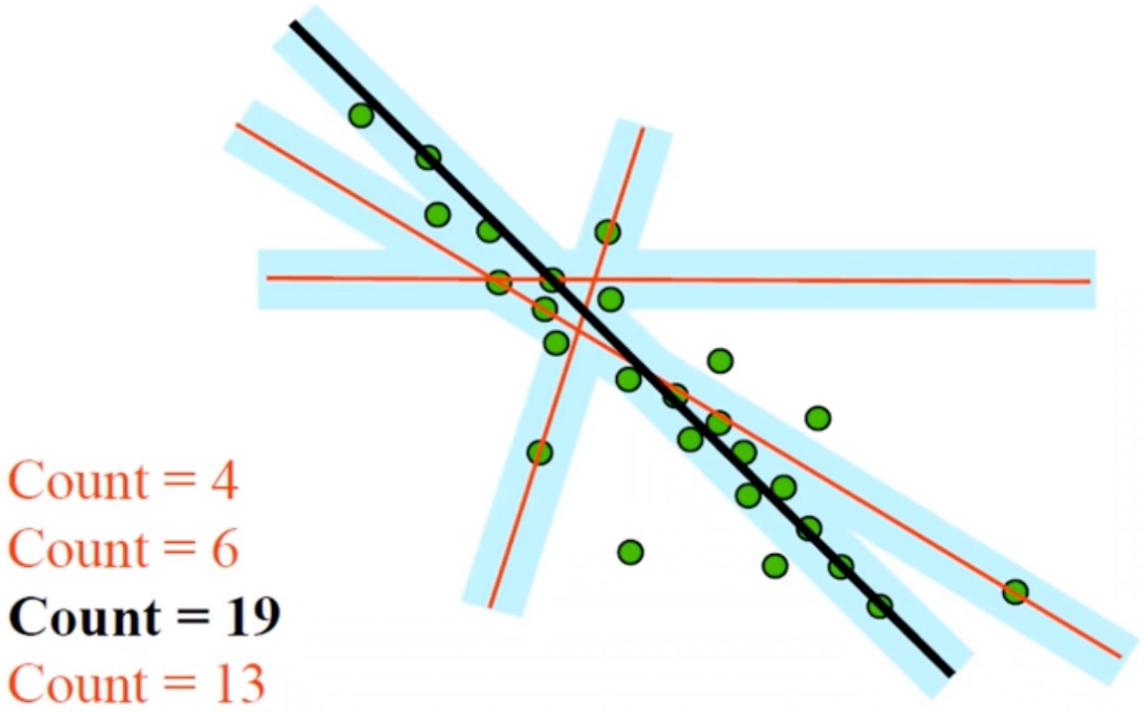
Outlier reduction using RANSAC

- Random Sample Consensus

Example task:
Given some points, find a line that fits them best

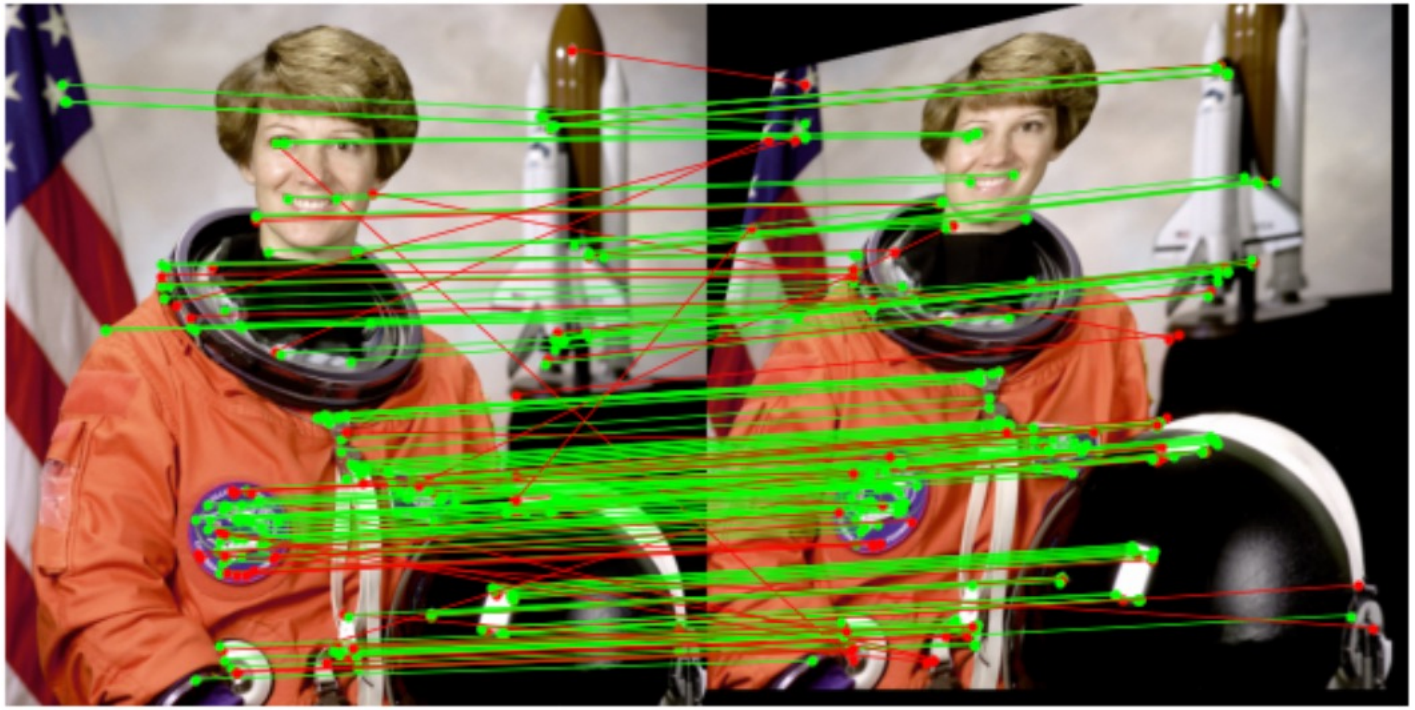
Algorithm:
1. sample two points randomly
2. fit a line y = kx + b
3. count how many points fall to this line with an allowed error d (inliers)
4. if inliers are more than our previous best, keep this model (line with params k and b)
4. repeat (N times)

Output: the line that has the most amount of inliers



Feature matching

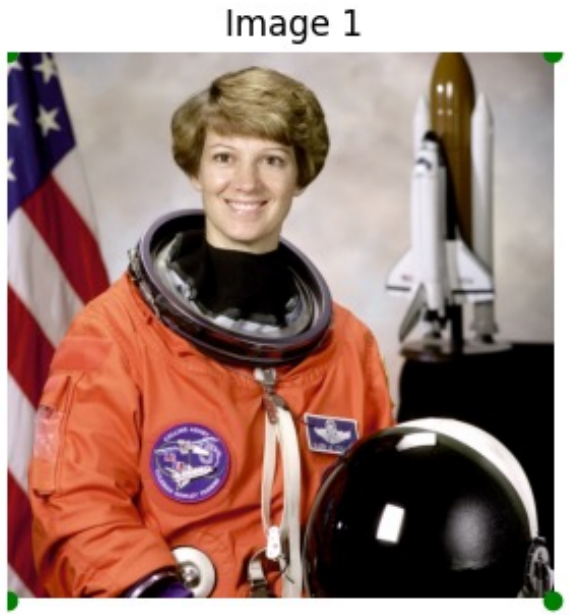
- Step-by-step feature matching using ORB

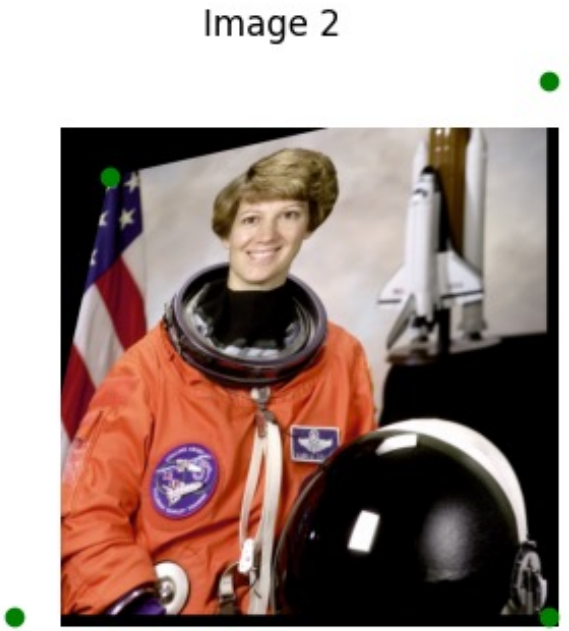


Setup

```
img1 = data.astronaut()
tform = transform.ProjectiveTransform()
tform.estimate(
    np.array([[0, 0], [512, 0], [512, 512], [0, 512]]),
    np.array([[50, 50], [500, -50], [500, 500], [-50, 500]])
)
img2 = transform.warp(img1, tform.inverse)
np.set_printoptions(precision=3, suppress=True)
print(tform.params)
```

```
[[ 0.701 -0.178 50.   ]
 [-0.178  0.701 50.   ]
 [-0.    -0.     1.   ]]
```





Homography – planar transformation

```
eye_coords1 = np.array([[100, 200], [100, 245]])
eye_coords2 = tform(eye_coords1[:, ::-1])[:, ::-1]  # swap to (x,y), transform, swap back to (row,col)
print("Eye coords in img1 (row, col):\n", eye_coords1)
print("Mapped eye coords in img2 (row, col):\n", eye_coords2)
```

```
Eye coords in img1 (row, col):
 [[100 200]
 [100 245]]
Mapped eye coords in img2 (row, col):
 [[ 94.7 193.1]
 [ 87.3 232.6]]
```

```
[[ 0.701 -0.178 50.   ]
 [-0.178  0.701 50.   ]
 [-0.    -0.     1.   ]]
```

```
H = tform.params
print(H)
p = np.array([100, 200])  # (x, y)
p_h = np.array([p[0], p[1], 1])
p_t1 = H @ p_h
p_t1 /= p_t1[2]
print(p_t1[:2])  # (x', y')
```

Image 1 with Eye Coords



Image 2 with Mapped Eye Coords



ORB in scikit-image

```
orb = feature.ORB(n_keypoints=500)
orb.detect_and_extract(gray1)
keypoints1, descriptors1 = orb.keypoints, orb.descriptors

orb.detect_and_extract(gray2)
keypoints2, descriptors2 = orb.keypoints, orb.descriptors
```

Image 1 Keypoints



Image 2 Keypoints



```
plt.imshow(img1)
plt.scatter(keypoints1[:, 1], keypoints1[:, 0], marker='o', color='r', s=10)
```

ORB in scikit-image

```
print(descriptors1.shape, descriptors2.shape)
print(descriptors1.dtype, descriptors2.dtype)

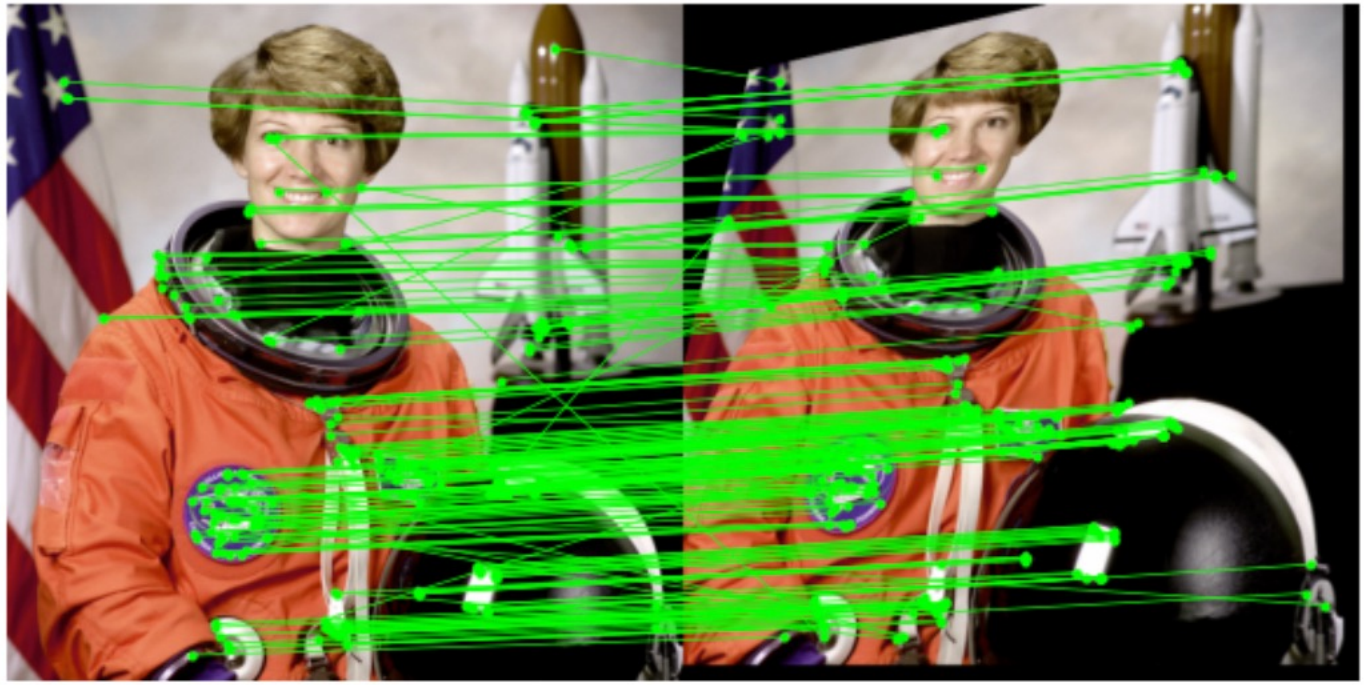
print(descriptors1[0][:16])
print(descriptors2[0][:16])
```

```
(500, 256) (500, 256)
bool bool
[False False False  True  True  True  True False False  True False False
  True  True False False]
[ True False False  True  True  True  True False False False False False
  True  True False False]
```

ORB match descriptors

```
matches12 = feature.match_descriptors(descriptors1, descriptors2, cross_check=True, metric='hamming')
print(f"Number of matches: {len(matches12)} / {min(len(descriptors1), len(descriptors2))}")
print(matches12[:5])  # Print first 5 matches
print(matches12.shape)
```

```
Number of matches: 251 / 500
[[ 1 14]
 [ 2 95]
 [ 4 32]
 [ 6 22]
 [ 8 31]]
(251, 2)
```



RANSAC

```
min_samples = 4
residual_threshold = 2
max_trials = 20  # keep it small for visualization
best_inliers = []
best_model = None

src = keypoints1[matches12[:, 0]]
dst = keypoints2[matches12[:, 1]]
```

Image 1 Keypoints



Image 2 Keypoints



```
for i in range(max_trials):
    # Randomly sample minimal set
    idx = random.sample(range(len(src)), min_samples)
    src_subset = src[idx]
    dst_subset = dst[idx]

    # Fit projective transform
    model = transform.ProjectiveTransform()
    if not model.estimate(src_subset, dst_subset):
        continue  # skip if estimation fails

    # Compute residuals for all matches
    projected = model(src)
    residuals = np.linalg.norm(projected - dst, axis=1)

    # Determine inliers
    inliers = residuals < residual_threshold

    # Update best model
    if inliers.sum() > len(best_inliers):
        best_inliers = inliers
        best_model = model
```

Iteration 1: Inliers 35



Iteration 2: Inliers 109



Iteration 3: Inliers 32



Iteration 4: Inliers 125



Iteration 5: Inliers 124



Iteration 6: Inliers 21



Iteration 7: Inliers 4



Iteration 8: Inliers 13



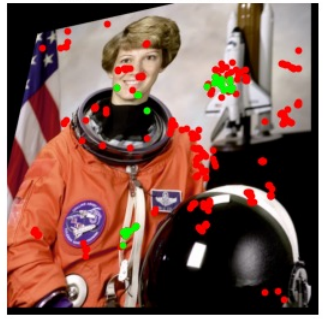
Iteration 9: Inliers 84



Iteration 10: Inliers 4



Iteration 11: Inliers 34



Iteration 12: Inliers 17



Iteration 13: Inliers 31



Iteration 14: Inliers 5



Iteration 15: Inliers 4



Iteration 16: Inliers 73



Iteration 17: Inliers 8



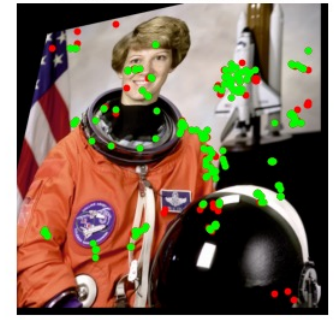
Iteration 18: Inliers 7



Iteration 19: Inliers 131



Iteration 20: Inliers 180



RANSAC in scikit.measure

```
src = keypoints1[matches12[:, 0]]
dst = keypoints2[matches12[:, 1]]

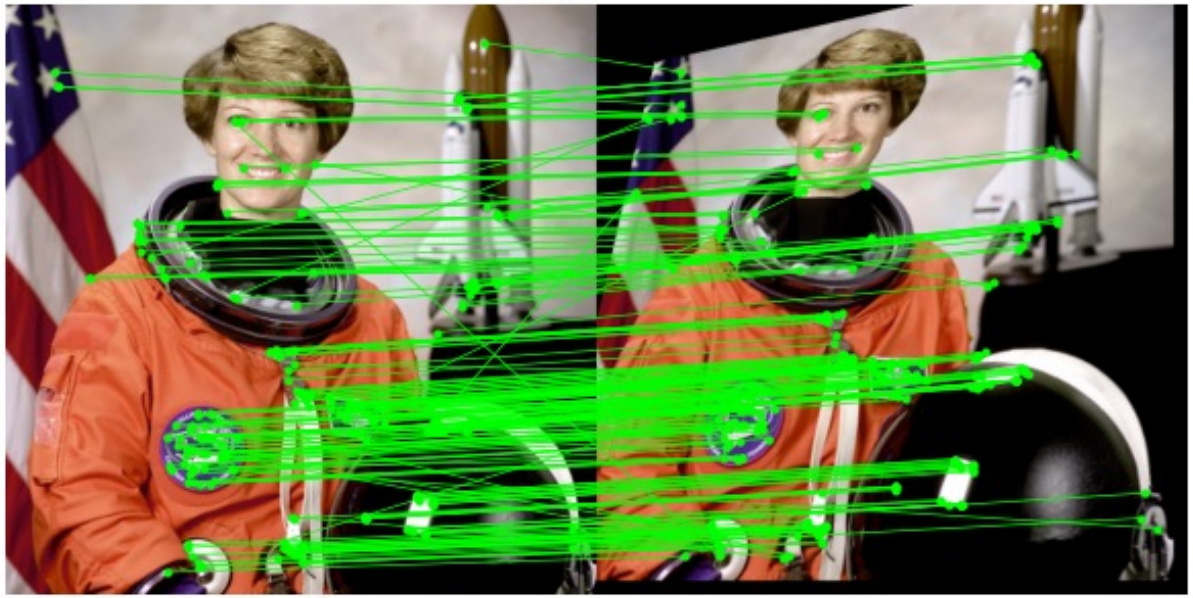
model_robust, inliers = ransac(
    (src, dst),
    transform.ProjectiveTransform,
    min_samples=4,
    residual_threshold=2,
    max_trials=1000
)
```

```
print(f"Estimated transformation:\n{model_robust.params}")
print(f"Ground truth transformation:\n{tform.params}")
```

```
Estimated transformation:
[[ 0.7 -0.2 49.7]
 [-0.2  0.7 49.5]
 [-0.  -0.   1. ]]
Ground truth transformation:
[[ 0.7 -0.2 50. ]
 [-0.2  0.7 50. ]
 [-0.  -0.   1. ]]
```

Outlier reduction using RANSAC

All Matches (Before RANSAC)



Inlier Matches (After RANSAC)



Transform img2 back to img1

```
img2_aligned = transform.warp(img2, model_robust, output_shape=img1.shape)
```

Image 1



Image 2 (Transformed)



Image 2 Aligned to Image 1



Image to image registration in real world

